

Experiment 1

Aim

To write a Pandas program to select the distinct department IDs from the employees file.

Procedure

1. Import the Pandas library.
2. Upload the departments.csv file in Google Colab.
3. Read the CSV file using `pd.read_csv()`.
4. Select the `DEPARTMENT_ID` column.
5. Display the distinct department IDs using `drop_duplicates()`.

CSV File (departments.csv)

DEPARTMENT_ID,DEPARTMENT_NAME,MANAGER_ID,LOCATION_ID

10,Administration,200,1700

20,Marketing,201,1800

30,Purchasing,114,1700

40,Human Resources,203,2400

50,Shipping,121,1500

60,IT,103,1400

70,Public Relations,204,2700

80,Sales,145,2500

90,Executive,100,1700

100,Finance,108,1700

110,Accounting,205,1700

120,Treasury,0,1700

130,Corporate Tax,0,1700

140,Control And Credit,0,1700

150,Shareholder Services,0,1700

160,Benefits,0,1700

170,Manufacturing,0,1700
180,Construction,0,1700
190,Contracting,0,1700
200,Operations,0,1700
210,IT Support,0,1700
220,NOC,0,1700
230,IT Helpdesk,0,1700
240,Government Sales,0,1700
250,Retail Sales,0,1700
260,Recruiting,0,1700
270,Payroll,0,1700

Google Colab Code

```
import pandas as pd
from google.colab import files

# Upload CSV file
uploaded = files.upload()

# Read CSV
df = pd.read_csv("departments.csv")

print("Employees Data")
print(df)

print("\nDistinct Department IDs")
print(df["DEPARTMENT_ID"].drop_duplicates())
```

Expected Output

Displays all unique Department IDs.

```
Distinct Department IDs:
0      10
1      20
2      30
3      40
4      50
5      60
6      70
7      80
8      90
9     100
10     110
11     120
12     130
13     140
14     150
15     160
16     170
17     180
18     190
19     200
20     210
21     220
22     230
23     240
24     250
25     260
26     270
```

Conclusion

Thus, the distinct department IDs were successfully displayed using the Pandas `drop_duplicates()` function.

Experiment 2

Aim

To write a Pandas program to display the IDs of employees who have done two or more jobs in the past.

Procedure

1. Import Pandas.
2. Upload the CSV file.
3. Read the CSV file.
4. Count the number of jobs done by each employee.
5. Display employee IDs having two or more jobs.

CSV File (job_history.csv)

EMPLOYEE_ID	START_DATE	END_DATE	JOB_ID	DEPARTMENT_ID
102	2001-01-13	2006-07-24	IT_PROG	60
101	1997-09-21	2001-10-27	AC_ACCOUNT	110
101	2001-10-28	2005-03-15	AC_MGR	110
201	2004-02-17	2007-12-19	MK_REP	20
114	2006-03-24	2007-12-31	ST_CLERK	50
122	2007-01-01	2007-12-31	ST_CLERK	50
200	1995-09-17	2001-06-17	AD_ASST	90
176	2006-03-24	2006-12-31	SA_REP	80
176	2007-01-01	2007-12-31	SA_MAN	80
200	2002-07-01	2006-12-31	AC_ACCOUNT	90

Google Colab Code

```
import pandas as pd

from google.colab import files

uploaded = files.upload()
```

```

df = pd.read_csv("job_history.csv")

print("Job History")

print(df)

job_count = df.groupby("EMPLOYEE_ID").size()

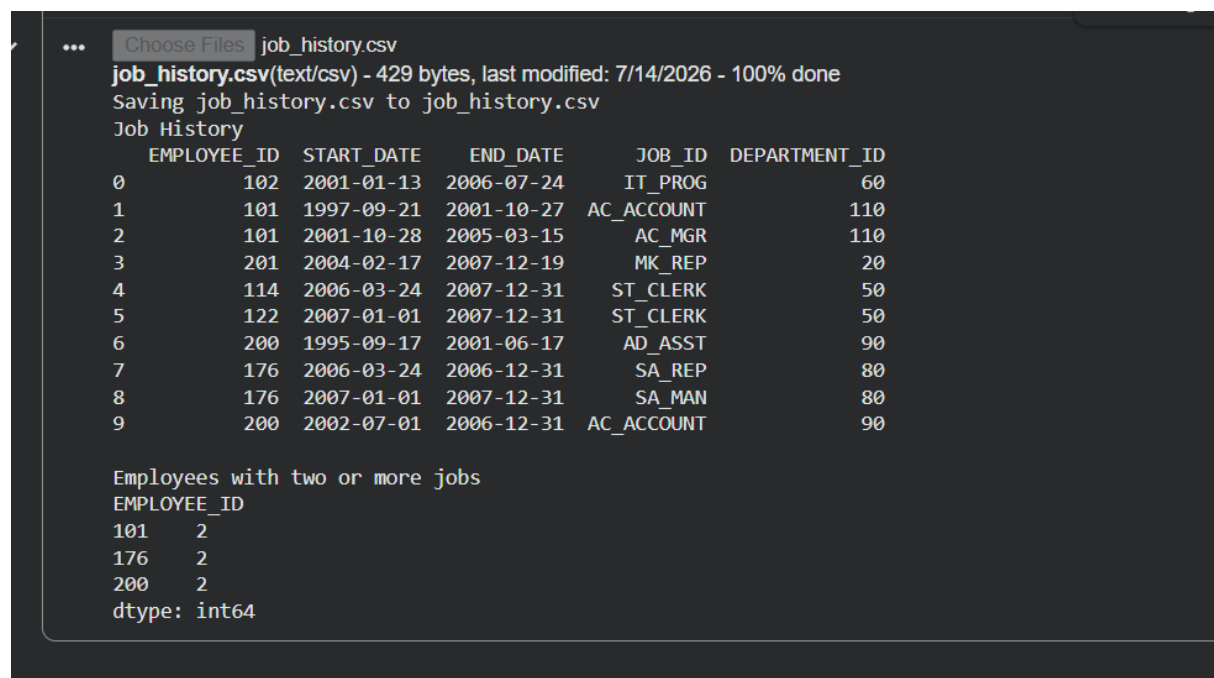
print("\nEmployees with two or more jobs")

print(job_count[job_count >= 2])

```

Expected Output

EMPLOYEE_ID



```

... Choose Files job_history.csv
job_history.csv(text/csv) - 429 bytes, last modified: 7/14/2026 - 100% done
Saving job_history.csv to job_history.csv
Job History
  EMPLOYEE_ID  START_DATE  END_DATE  JOB_ID  DEPARTMENT_ID
0          102  2001-01-13  2006-07-24   IT_PROG             60
1          101  1997-09-21  2001-10-27  AC_ACCOUNT             110
2          101  2001-10-28  2005-03-15   AC_MGR             110
3          201  2004-02-17  2007-12-19   MK_REP             20
4          114  2006-03-24  2007-12-31  ST_CLERK             50
5          122  2007-01-01  2007-12-31  ST_CLERK             50
6          200  1995-09-17  2001-06-17  AD_ASST             90
7          176  2006-03-24  2006-12-31  SA_REP             80
8          176  2007-01-01  2007-12-31  SA_MAN             80
9          200  2002-07-01  2006-12-31  AC_ACCOUNT             90

Employees with two or more jobs
EMPLOYEE_ID
101      2
176      2
200      2
dtype: int64

```

Conclusion

Thus, the employees who have worked in two or more jobs were successfully identified using the `groupby()` function.

Experiment 3

Aim

To write a Pandas program to display the details of jobs in descending order of Job Title.

Procedure

1. Import Pandas.
2. Upload the CSV file.
3. Read the CSV.
4. Sort the Job Title column in descending order.
5. Display the sorted DataFrame.

CSV File (jobs.csv)

JOB_ID,JOB_TITLE,MIN_SALARY,MAX_SALARY

AD_PRES,President,20080,40000

AD_VP,Administration Vice President,15000,30000

AD_ASST,Administration Assistant,3000,6000

FI_MGR,Finance Manager,8200,16000

FI_ACCOUNT,Accountant,4200,9000

AC_MGR,Accounting Manager,8200,16000

AC_ACCOUNT,Public Accountant,4200,9000

SA_MAN,Sales Manager,10000,20080

SA_REP,Sales Representative,6000,12008

PU_MAN,Purchasing Manager,8000,15000

PU_CLERK,Purchasing Clerk,2500,5500

ST_MAN,Stock Manager,5500,8500

ST_CLERK,Stock Clerk,2008,5000

SH_CLERK,Shipping Clerk,2500,5500

IT_PROG,Programmer,4000,10000

MK_MAN,Marketing Manager,9000,15000
MK_REP,Marketing Representative,4000,9000
HR_REP,Human Resources Representative,4000,9000
PR_REP,Public Relations Representative,4500,10500

Google Colab Code

```
import pandas as pd
from google.colab import files

uploaded = files.upload()

df = pd.read_csv("jobs.csv")

print("Jobs Data")
print(df)

sorted_df = df.sort_values(by="JOB_TITLE", ascending=False)

print("\nJobs in Descending Order of Job Title")

print(sorted_df)
```

Expected Output

The jobs are displayed in descending alphabetical order based on the **JOB_TITLE** column.

13	SH_CLERK	Shipping Clerk	2500	5500
14	IT_PROG	Programmer	4000	10000
15	MK_MAN	Marketing Manager	9000	15000
16	MK_REP	Marketing Representative	4000	9000
17	HR_REP	Human Resources Representative	4000	9000
18	PR_REP	Public Relations Representative	4500	10500

Jobs in Descending Order of Job Title				
	JOB_ID	JOB_TITLE	MIN_SALARY	MAX_SALARY
11	ST_MAN	Stock Manager	5500	8500
12	ST_CLERK	Stock Clerk	2008	5000
13	SH_CLERK	Shipping Clerk	2500	5500
8	SA_REP	Sales Representative	6000	12008
7	SA_MAN	Sales Manager	10000	20080
9	PU_MAN	Purchasing Manager	8000	15000
10	PU_CLERK	Purchasing Clerk	2500	5500
18	PR_REP	Public Relations Representative	4500	10500
6	AC_ACCOUNT	Public Accountant	4200	9000
14	IT_PROG	Programmer	4000	10000
0	AD PRES	President	20080	40000
16	MK_REP	Marketing Representative	4000	9000
15	MK_MAN	Marketing Manager	9000	15000
17	HR_REP	Human Resources Representative	4000	9000
3	FI_MGR	Finance Manager	8200	16000
1	AD_VP	Administration Vice President	15000	30000
2	AD_ASST	Administration Assistant	3000	6000
5	AC_MGR	Accounting Manager	8200	16000
4	FI_ACCOUNT	Accountant	4200	9000

Conclusion

Thus, the job details were successfully sorted and displayed in descending order using the Pandas `sort_values()` function.

Experiment 4

Aim

To create a line plot of the historical stock prices of Alphabet Inc. between two specific dates.

Procedure

1. Import the required libraries.
2. Upload the `alphabet_stock_data.csv` file.
3. Read the CSV using Pandas.
4. Convert the Date column to datetime format.
5. Plot the closing stock prices using a line chart.

Google Colab Code

```
import pandas as pd
import matplotlib.pyplot as plt
from google.colab import files

# Upload CSV file
uploaded = files.upload()

# Read CSV
df = pd.read_csv("alphabet_stock_data.csv")

# Convert Date column
df["Date"] = pd.to_datetime(df["Date"])

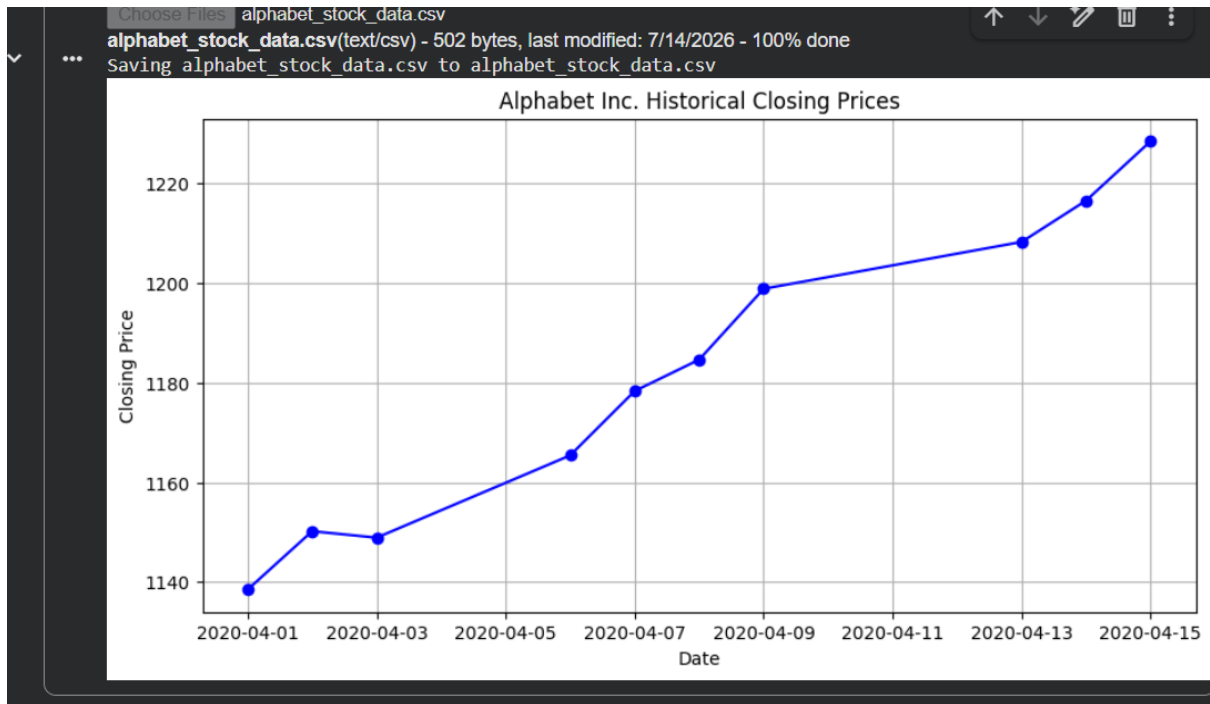
# Line Plot
plt.figure(figsize=(10,5))
plt.plot(df["Date"], df["Close"], marker="o", color="blue")

plt.title("Alphabet Inc. Historical Closing Prices")
plt.xlabel("Date")
plt.ylabel("Closing Price")
plt.grid(True)
```

```
plt.show()
```

Output

A **line graph** showing the variation in Alphabet Inc.'s closing stock prices over the selected dates.



Conclusion

Thus, the historical closing stock prices of Alphabet Inc. were successfully visualized using a line plot.